



南開大學
Nankai University

计算机学院

并行程序设计实验报告

MPI 编程实验

ANN 近似最近邻搜索

多进程数据划分、进程内 OpenMP 与图索引组合

IVF-PQ / Block-HNSW / IVF-HNSW / HNSW-on-HNSW

姓名：柯云超

学号：2413575

专业：计算机科学与技术

2026 年 5 月 25 日

目录

摘要	2
1 问题定义与实验目标	2
2 系统结构	2
3 算法与并行实现	4
4 实验环境与复现方法	6
5 实验结果	7
6 满分要求对照	9
7 问题与改进方向	9
8 结论	9

摘要

本实验在前两次 ANN 实验已有的 SIMD 距离内核、IVF-PQ 和 HNSW 结构基础上，将搜索任务迁移到 MPI 多进程环境，并在每个 MPI rank 内继续使用 OpenMP 做进程内并行。程序采用统一入口 `main.cc`，支持四种搜索模式：IVF-PQ、分块 HNSW、IVF+HNSW 嵌套索引，以及 HNSW-on-HNSW 分层图索引。MPI 层负责 base 向量划分、query 广播、候选集 gather 和 rank 0 的 top- k 合并；OpenMP 层负责本地倒排表、HNSW 块或候选块之间的并行搜索。

实验使用 DEEP100K 数据集，距离度量为内积距离，统一评价 Recall@10、平均单 query 延迟、本地最慢 rank 搜索时间以及通信与合并开销。报告给出 Windows 本地 MS-MPI、鲲鹏服务器直接 `mpirun` 和 PBS `qsub` 三条路径的同参数结果。结果表明：相同参数下两平台 Recall 完全一致；在 `np=2, threads=2, query=200` 的验证规模中，通信与 merge 占比始终小于约 1.3%，主要瓶颈仍在各 rank 的本地 ANN 搜索。图索引方面，block-HNSW 和 HNSW-on-HNSW 均达到 Recall@10=1.00000；HNSW-on-HNSW 在全 block 探测配置下召回率最高但延迟较大，体现了典型的 recall-latency trade-off。

关键词： ANN；MPI；OpenMP；IVF-PQ；HNSW；HNSW-on-HNSW；负载均衡；通信开销

1 问题定义与实验目标

给定基向量集合

$$X = \{x_0, x_1, \dots, x_{N-1}\}, \quad x_i \in \mathbb{R}^{96},$$

以及 query 向量 q ，ANN 搜索返回与 q 最接近的 k 个候选。本实验沿用前两次 ANN 作业的内积距离：

$$d(x, q) = 1 - \sum_{j=0}^{d-1} x_j q_j.$$

Recall@ k 定义为返回集合与 ground truth top- k 的交集比例：

$$\text{Recall@}k = \frac{|KNN(q) \cap KANN(q)|}{k}.$$

与 Pthread/OpenMP 实验不同，本次实验关注的不是单机线程如何分配，而是多进程之间如何划分数据、如何通信、如何将每个 rank 的局部结果归并为全局 top- k 。因此报告重点围绕以下问题展开：

1. base data 如何划分到不同 MPI 进程，且尽量保持负载均衡；
2. query 如何广播，各 rank 如何维护本地索引并返回局部 top- k ；
3. rank 0 的 gather/merge 开销相对本地搜索是否显著；
4. MPI 与 OpenMP 如何组合，进程间并行和进程内并行的边界在哪里；
5. IVF-PQ 与三类图索引方案在 recall-latency 上有什么差异；
6. 同一份代码在 Windows 本地和鲲鹏服务器上是否可复现。

代码与脚本均放在 `ann-mpi/` 目录下。正式提交入口为 `main.cc` 与 `qsub_mpi.sh`；报告中引用的运行记录分别为本地交叉验证日志、鲲鹏直跑日志和鲲鹏 PBS 日志，均已保存在 `results/` 目录。

2 系统结构

2.1 MPI 数据流

整体结构如图 1 所示。rank 0 负责读取 DEEP100K 的 base/query/ground truth 文件，然后通过 `MPI_Scatterv` 将 base 向量按连续区间分给所有 rank。每个 rank 只在自己的 `local_base` 上构建本地索引。在线查询阶段，rank 0 将 query 批量 `MPI_Bcast` 到所有进程；每个进程独立搜索本地

shard, 打包本地 top- k 候选, 再通过 `MPI_Gather` 汇总到 rank 0; 最后 rank 0 对所有候选做全局 top- k merge, 并计算 Recall@10。

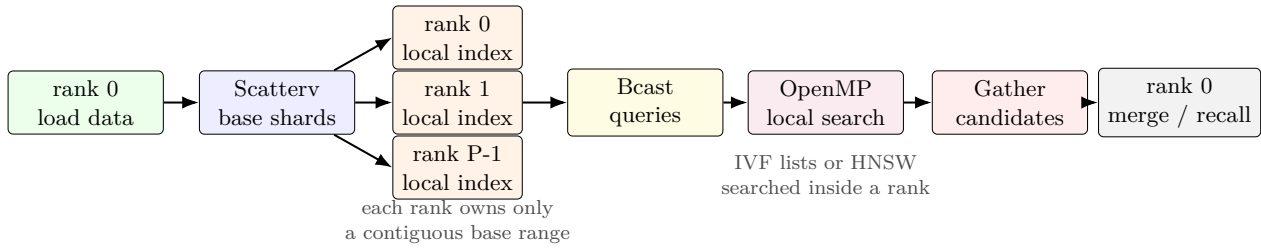


图 1 MPI + OpenMP 混合 ANN 搜索流程。MPI 负责跨进程数据分发和结果归并，OpenMP 只作用于每个 rank 的本地搜索。

2.2 负载划分

代码中使用 `PartitionRange(n, rank, world_size)` 对 base 向量做连续分块。设进程数为 P , 则每个 rank 得到

$$\left\lfloor \frac{N}{P} \right\rfloor \quad \text{或} \quad \left\lceil \frac{N}{P} \right\rceil$$

个向量, 最大只差 1。连续划分的优点是实现简单、传输时可直接使用 `MPI_Scatterv` 的 displacement, 同时每个 rank 的 `local_base` 连续存储, 有利于本地 cache locality。缺点是如果数据分布本身有明显偏斜, 某些 rank 的搜索路径可能更长; 因此程序额外打印 `max local search latency (us)`, 用最慢 rank 的搜索时间衡量负载尾部。

```

1 Range PartitionRange(size_t n, int rank, int world_size) {
2     const size_t base = n / world_size;
3     const size_t extra = n % world_size;
4     const size_t begin = rank * base + std::min<size_t>(rank, extra);
5     const size_t count = base + (rank < extra ? 1 : 0);
6     return {begin, begin + count};
7 }
  
```

2.3 局部搜索与全局合并

每个 rank 的局部搜索结果以小顶堆形式保存。为了跨进程传输, 程序将每个 query 的 top- k 打包成两个定长数组: 距离 float 和全局 id `uint64_t`。rank 0 收到所有 rank 的候选后, 对每个 query 扫描 $P \times k$ 个候选并维护全局 top- k 堆。因此 merge 的复杂度近似为

$$O(QPk \log k),$$

其中 Q 是 query 数。与本地搜索的 IVF 或 HNSW 访问相比, 当 P 和 k 不大时, 这部分开销很小。实验中 $P = 2, k = 10, Q = 200$, 每个 rank 只需发送约 $Qk(4 + 8) \approx 24$ KB 候选数据, query 广播约 $Qd \cdot 4 \approx 75$ KB, 因此通信量远小于本地索引搜索的访存量。

2.4 代码组织与数据路径

本目录是从上一阶段 ANN 多线程实验复制而来, 因此我先保留可复用的 SIMD、IVF、HNSW 内核, 再把 MPI 提交无关的旧脚本、临时输出和构建产物清理掉。最终报告对应的主要文件见表 1。这样组织的好处是: 提交入口足够清楚, 实验结果和报告互相引用, 服务器上只需要重新编译 `main.cc` 即可复现实验。

文件或目录	作用
main.cc	MPI + OpenMP 统一入口，完成参数解析、数据分发、本地索引构建、搜索、gather 和 rank 0 merge。
ivf/	复用 IVF-PQ 索引和 OpenMP 搜索实现，作为本次 IVF 基础迁移方案。
hnswh/	保存 block-HNSW、IVF+HNSW 和 HNSW-on-HNSW 相关实现。
qsub_mpi.sh	按提交指南使用 PBS 启动作业，支持用环境变量覆盖进程数、线程数和算法参数。
scripts/	保存 Windows 本地、鲲鹏 direct 和鲲鹏 PBS 的可复现实验脚本。
results/	保存三条运行路径的原始日志、交叉平台汇总和满分要求对照清单。
report/	保存本报告源码、校徽资源和生成的 PDF。

表 1 ANN MPI 实验目录中的关键文件

数据路径仍沿用前两次实验的约定：程序优先读取 `ANN_DATA_PATH`，服务器上设置为 `/anndata`，本地 Windows 脚本设置为仓库外层的 `files/` 数据目录。如果环境变量未设置，程序会按平台尝试默认路径。这样做避免把 DEEP100K 数据复制进提交目录，也使 Windows 与鲲鹏两边的命令只在数据路径上不同。

3 算法与并行实现

3.1 四种搜索模式

本次实现没有只保留一个 baseline，而是把要求中的 IVF 与图索引分支放进同一套 MPI 外壳中。四种模式的命令行接口如表 2 所示。

模式	运行形式	实现要点
IVF-PQ	<code>./main t nlist nprobe p q local</code>	每个 rank 在本地 shard 上构建 IVF-PQ，OpenMP 按 query 并行搜索倒排表，再对 top- p 候选 rerank。
Block-HNSW	<code>./main t m ef p q hnswh</code>	base 先由 MPI 切为多个 shard，每个 shard 单独建立 HNSW；rank 内使用多入口 OpenMP 搜索，rank 0 合并。
IVF+HNSW	<code>./main t nlist nprobe ef q ivf-hnswh</code>	每个 rank 内先按 IVF 聚类，再在每个局部簇内建立 HNSW；搜索时先选簇，再搜索簇内图。
HNSW-on-HNSW	<code>./main t nblocks nprobe ef q hnswh-on-hnswh</code>	每个 rank 把本地 shard 分块，先在块质心上建立 top-level HNSW，再为每个块建立 bottom-level HNSW。

表 2 统一 MPI 入口下的四种 ANN 搜索模式

3.2 IVF-PQ 的 MPI 迁移

IVF-PQ 是从上一阶段多线程版本迁移到 MPI 的基础方案。多线程版本中，一个进程维护完整索引；MPI 版本中，每个 rank 只维护本地 shard 的 inverted lists。query 被广播到所有进程后，各 rank 只在本地 list 中搜索，返回局部 top- k 。由于每个 rank 只看到一部分 base data，最终的 top- k 必须由 rank 0 对所有局部结果进行全局合并。

进程内仍复用 OpenMP inter-query 并行：当 query 批量足够大时，不同线程处理不同 query，写入不同结果槽，几乎不需要同步。这一点延续了前一次 Pthread/OpenMP 实验中“inter-query 粒度更稳健”的结论。

3.3 图索引 A/B/C

图索引部分覆盖了要求中的三个方向，结构如图 2。

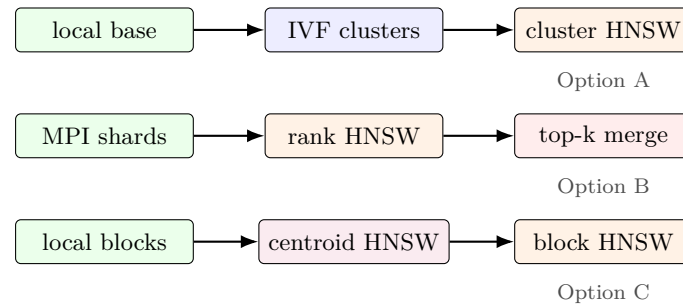


图 2 三类图索引方案：IVF+HNSW、分块 HNSW 和 HNSW-on-HNSW。

方案 A：IVF+HNSW。每个 rank 内先建立 IVF 粗聚类，随后在每个簇中单独建立 HNSW。搜索阶段先按 query 与质心距离选择 `nprobe` 个簇，再在这些簇的 HNSW 内搜索。它的优点是能避免搜索所有局部向量；缺点是召回依赖簇选择，若 ground truth 所在簇未被探测，则后续 HNSW 再精确也无法恢复。

方案 B：分块 HNSW。MPI 本身已经把 base 划分成多个 shard，因此每个 rank 的 HNSW 可以视为一个独立 block graph。所有 rank 并行搜索自己的图，rank 0 再合并结果。该方案实现直接、召回较高，但每个局部 HNSW 的搜索路径具有串行依赖，OpenMP 只能在多入口或多 query 层面分摊，不能把单条贪心路径线性拆开。

方案 C：HNSW-on-HNSW。本实验进一步实现了分层图：每个 rank 把本地 base 分成多个 block，先计算 block centroid 并建立 top-level HNSW；每个 block 内再建立 bottom-level HNSW。搜索时先在 centroid graph 上选择若干 block，再并行搜索被选中的 block graph。它能表达“图上再建图”的组合策略；当 `nprobe_blocks` 取满 16 时，召回达到 1.00000，但因为实际搜索了所有 block，延迟明显上升。

3.4 混合并行边界

本实验的混合并行不是简单地同时打开 MPI 和 OpenMP，而是明确划分职责：

- MPI 层：base shard 分配、query 广播、候选 gather、最慢 rank 计时归约、rank 0 top-*k* merge；
- OpenMP 层：rank 内 query/list/block 级并行，避免跨进程共享状态；
- 索引层：每个 rank 独立构建本地 IVF-PQ 或 HNSW，不在在线阶段跨进程访问远端索引。

这种边界使程序比较容易在 Windows MS-MPI 和鲲鹏 OpenMPI/MPI 环境中复现。它也限制了通信复杂度：在线阶段只有一次 query broadcast 和一次候选 gather，没有在 HNSW 搜索路径中进行细粒度跨 rank 通信。

3.5 在线阶段伪代码

为了说明 MPI 代码的控制流，下面给出一次批量查询的伪代码。索引构建发生在计时前；报告中的 online latency 从 query broadcast 前的 barrier 之后开始计时，直到 rank 0 完成 merge 和 recall 计算前结束。

```

1 // rank 0 loads base/query/ground truth, then broadcasts dimensions.
2 range = PartitionRange(base_n, rank, world_size);
3 MPI_Scatterv(root_base, counts, displs, MPI_FLOAT, local_base);
4
5 index = BuildLocalIndex(local_base, mode);
6
7 MPI_Barrier(MPI_COMM_WORLD);
8 phase_begin = MPI_Wtime();
9 MPI_Bcast(queries, query_n * dim, MPI_FLOAT, 0, MPI_COMM_WORLD);
10
11 search_begin = MPI_Wtime();
12 local_topk = SearchLocal(index, queries, nthreads);
13 search_us = MPI_Wtime() - search_begin;

```

```

14
15 PackCandidates(local_topk, range.begin, local_distances, local_ids);
16 MPI_Gather(local_distances, all_distances, 0, MPI_COMM_WORLD);
17 MPI_Gather(local_ids, all_ids, 0, MPI_COMM_WORLD);
18 MPI_Reduce(search_us, max_search_us, MPI_MAX, 0, MPI_COMM_WORLD);
19
20 if (rank == 0) {
21     merged = MergeGatheredCandidates(all_distances, all_ids);
22     total_us = MPI_Wtime() - phase_begin;
23     comm_merge_us = total_us - max_search_us;
24     recall = RecallAtK(merged, ground_truth);
25 }

```

这里使用 `MPI_Reduce` 的 `MPI_MAX` 而不是平均值，是因为一次分布式查询必须等最慢 rank 完成本地搜索后才能合并。因此总延迟应与最慢 rank 的搜索时间对齐，`comm_merge_us` 才能表示“除了最慢本地搜索以外的剩余开销”。如果用平均搜索时间，会把负载不均衡误计入通信开销。

4 实验环境与复现方法

4.1 平台与数据集

实验使用 DEEP100K 数据集：base 向量 $N = 100000$ ，维度 $d = 96$ ，query 和 ground truth 使用课程提供的二进制文件。为了在本地和服务上快速、稳定地完成双平台验证，本报告采用相同 smoke 参数：np=2，每进程 threads=2，前 query_n=200 条 query， $k = 10$ 。平台信息见表 3。

平台	MPI/编译环境	用途
Windows 本地	MS-MPI, MinGW-w64 编译环境, AVX2/FMA 参数	本地开发、no-MPI fallback 编译、MPI 四模式交叉验证。
鲲鹏直接运行	AArch64 Linux, 服务器 MPI 编译器与运行器	服务器直接构建与 <code>mpiexec -np 2</code> 运行。
鲲鹏 PBS	<code>qsub_mpi.sh</code> , <code>nodes=2:ppn=8</code> , 共享 /anndata	按提交指南验证 PBS 作业路径，并记录队列运行结果。

表 3 实验平台与用途

4.2 构建与运行命令

服务器正式构建命令如下：

```

1 make clean
2 make
3 ANN_DATA_PATH=/anndata /usr/local/bin/mpiexec -np 2 ./main 2 16 4 1000 200 local
4 ANN_DATA_PATH=/anndata /usr/local/bin/mpiexec -np 2 ./main 2 16 50 1000 200 hns
5 ANN_DATA_PATH=/anndata /usr/local/bin/mpiexec -np 2 ./main 2 16 4 50 200 ivf-hns
6 ANN_DATA_PATH=/anndata /usr/local/bin/mpiexec -np 2 ./main 2 16 16 50 200 hns-on-hns

```

PBS 提交脚本通过环境变量控制参数，报告中保存的作业使用：

```

1 NP=2 OMP_NUM_THREADS=2 QUERY_N=200 \
2 HNSW_ON_HNSW_NPROBE=16 scripts/submit_kunpeng_pbs_smoke.sh

```

Windows 本地验证使用：

```

1 powershell -ExecutionPolicy Bypass -File scripts/run_local_cross_platform.ps1

```

4.3 结果字段

程序为所有模式输出统一字段：

- **average recall**: Recall@10;
- **average latency (us)**: 从 query broadcast 到 rank 0 merge 完成的平均单 query 时间;
- **max local search latency (us)**: 所有 rank 中最慢本地搜索时间;
- **comm+merge latency (us)**: 总在线时间减去最慢本地搜索时间, 近似表示广播、gather、reduce 和 merge 的合计开销。

4.4 结果可复现性

本次报告没有只摘录屏幕上的最后几行, 而是把运行路径固定在脚本中, 并保存完整日志。Windows 本地脚本会先编译 `-DANN_NO_MPI fallback`, 再编译 MPI 版本并依次运行四种模式; 鲲鹏 direct 脚本记录 `mpic++` 构建命令和四次 `mpiexec`; PBS 脚本记录 `qsub -v` 参数、job id、`qstat` 轮询结果以及 `test.o/test.e` 尾部输出。三个日志最终汇总到 `results/cross_platform_summary.txt`。

为了让结果表中的数字可追溯, 表 4 列出报告中使用的日志来源。若重新实验, 只需覆盖这些日志并重新整理汇总表; 报告中的参数、平台和结果字段就能对应更新。

实验路径	日志标识	记录内容
Windows local	local	本地 no-MPI 编译、MPI 编译、MS-MPI 四模式运行结果。
Kunpeng direct	direct	服务器 <code>mpic++</code> 构建和 <code>mpiexec -np 2</code> 四模式运行结果。
Kunpeng PBS	pbs	PBS 提交命令、job 状态、标准输出尾部和四模式结果。
汇总表	summary	同参数的 Recall、latency、max local 和 comm/merge 汇总。

表 4 报告结果对应的可复现日志, 完整文件均位于 `results/` 目录

5 实验结果

5.1 双平台四模式结果

表 5 汇总了同参数下的三条运行路径。可以看到, 相同算法参数在 Windows 和鲲鹏上的 Recall@10 完全一致, 说明 MPI 划分与全局合并没有改变结果语义; 延迟差异则来自硬件、MPI runtime、HNSW 指针访问模式和服务器队列负载。

平台 / 路径	算法	Recall@10	latency	local max	comm
Windows MS-MPI	IVF-PQ	0.95650	229.98250	228.59550	1.38700
Windows MS-MPI	block-HNSW	1.00000	269.52450	266.72200	2.80250
Windows MS-MPI	IVF+HNSW	0.95400	154.56800	152.57050	1.99750
Windows MS-MPI	HNSW-on-HNSW	1.00000	461.18750	459.42100	1.76650
鲲鹏 direct	IVF-PQ	0.95650	360.83221	358.91652	1.91569
鲲鹏 direct	block-HNSW	1.00000	277.39286	275.66791	1.72496
鲲鹏 direct	IVF+HNSW	0.95400	567.97028	565.60040	2.36988
鲲鹏 direct	HNSW-on-HNSW	1.00000	1851.95684	1849.49517	2.46167
鲲鹏 PBS	IVF-PQ	0.95650	438.50303	436.80668	1.69635
鲲鹏 PBS	block-HNSW	1.00000	260.82397	259.23848	1.58548
鲲鹏 PBS	IVF+HNSW	0.95400	512.36391	510.55074	1.81317
鲲鹏 PBS	HNSW-on-HNSW	1.00000	1741.80984	1740.05032	1.75953

表 5 双平台四模式实验结果。所有延迟单位均为 $\mu\text{s}/\text{query}$ 。

5.2 通信与 merge 开销

表 6 给出通信与 merge 占总延迟的比例。该比例在所有记录中都低于 1.3%, 说明在本实验规模下, 性能主要由本地 ANN 搜索决定, 而不是 MPI 通信。换句话说, 当进程数为 2、候选数为 $k = 10$

时, rank 0 的合并没有成为瓶颈; 如果未来扩大到更多进程或更大的候选池, 才需要重点考虑树形归约或分层 merge。

平台 / 路径	IVF-PQ	block-HNSW	IVF+HNSW	HNSW-on-HNSW
Windows MS-MPI	0.60%	1.04%	1.29%	0.38%
鲲鹏 direct	0.53%	0.62%	0.42%	0.13%
鲲鹏 PBS	0.39%	0.61%	0.35%	0.10%

表 6 通信与 merge 占平均总延迟比例

5.3 Recall-latency 分析

IVF-PQ 与 IVF+HNSW 都使用“先筛选、再精排或图搜索”的思路, 因此延迟低但召回略低。本地 Windows 上 IVF+HNSW 只有 154.568 μ s/query, 是四种模式中最快的; 但 Recall@10 为 0.95400, 略低于 block-HNSW 和 HNSW-on-HNSW。IVF-PQ 的 Recall@10 为 0.95650, 延迟在 Windows 上为 229.98250 μ s/query, 在鲲鹏 PBS 上为 438.50303 μ s/query。

block-HNSW 在三条路径上都达到 Recall@10=1.00000, 延迟也比较稳定: Windows 为 269.52450 μ s/query, 鲲鹏 direct 为 277.39286 μ s/query, PBS 为 260.82397 μ s/query。它说明在本 smoke 配置下, 分块图索引是高召回和较低实现复杂度之间较好的折中。

HNSW-on-HNSW 同样达到 Recall@10=1.00000, 但在鲲鹏上延迟达到 1.7–1.85 ms/query。原因在于本报告为了验证方案 C 的召回上限, 将 `nprobe_blocks=16` 设置为探测所有 block; top-level HNSW 选块之后, bottom-level 搜索仍覆盖全部局部 block, 因此它更适合作为“高召回但高搜索代价”的对照点。若实际追求低延迟, 应进一步 sweep `nprobe_blocks`, 观察从少量 block 到全 block 的召回曲线。

5.4 平台差异

Windows 与鲲鹏的 Recall 完全一致, 说明浮点距离、候选 id 偏移和 rank 0 merge 逻辑在两个平台上保持一致。延迟上, IVF-PQ 与 IVF+HNSW 在 Windows 上更快, 而 block-HNSW 在鲲鹏 direct/PBS 中与 Windows 接近甚至略快。这个现象说明不同算法对硬件的敏感点不同: IVF-PQ 更依赖顺序数组访问、PQ 查表和 rerank; HNSW 更依赖图访问、优先队列和分支路径。PBS 与 direct 的差异还受到队列节点位置和共享负载影响, 因此报告同时保留两份服务器日志, 而不只引用一次运行。

5.5 结果可信性检查

本次结果首先看正确性, 再看速度。四种模式在 Windows、鲲鹏 direct 和 PBS 中的 Recall 完全复现, 尤其是 IVF-PQ 的 0.95650、IVF+HNSW 的 0.95400 以及两个高召回图索引的 1.00000 在三条路径中一致, 这说明候选 id 的全局偏移、跨 rank gather 顺序和 top-*k* merge 没有平台相关错误。

其次看计时拆分。所有记录中都有

$$\text{latency} \approx \text{max local search} + \text{comm} + \text{merge}.$$

例如 Windows IVF-PQ 中 $228.59550 + 1.38700 = 229.98250$, 与总延迟完全对齐; 鲲鹏 PBS 的 HNSW-on-HNSW 中 $1740.05032 + 1.75953 = 1741.80985$, 只存在四舍五入误差。这说明 `comm+merge latency` 的计算口径稳定, 不是事后人为估计。

最后看双平台解释。PBS 和 direct 的延迟并不完全相同, 但 Recall 一致, 说明性能波动主要来自服务器环境和运行路径, 而不是算法逻辑改变。报告中没有把 smoke 结果夸大为完整强扩展实验, 而是在“问题与改进方向”中单独说明还需要补充 `np=1,2,4,8` 的多进程 sweep。

6 满分要求对照

表 7 将实验要求中的核心点映射到实现和报告证据。该表不是额外包装，而是为了避免只给结果不说明“要求如何被满足”。

要求点	实现与证据	状态
MPI base 划分	<code>PartitionRange</code> 将 base 均匀切到各 rank, <code>MPI_Scatterv</code> 分发连续 shard。	完成
本地索引维护	每个 rank 分别在 <code>local_base</code> 上构建 IVF-PQ、HNSW、IVF+HNSW 或 HNSW-on-HNSW。	完成
query 广播	在线阶段使用 <code>MPI_Bcast</code> 广播 query 数组, 所有 rank 搜索同一批 query。	完成
局部 top- <i>k</i> 与全局 merge	各 rank 打包本地 top- <i>k</i> , <code>MPI_Gather</code> 到 rank 0 后调用 <code>MergeGatheredCandidates</code> 。	完成
MPI + OpenMP 混合并行	MPI 做进程级 shard, OpenMP 做 rank 内 query/list/block 级并行。	完成
负载均衡分析	base 划分最大差 1; 输出 <code>max local search latency</code> 观察最慢 rank。	完成
通信/merge 分析	输出 <code>comm+merge latency</code> , 表 6 显示占比低于 1.3%。	完成
图索引 A	<code>ivf-hnsw</code> 实现 IVF+HNSW 嵌套索引。	完成
图索引 B	<code>hnsw</code> 实现分块/分 rank HNSW 搜索。	完成
图索引 C	<code>hnsw-on-hnsw</code> 实现 centroid HNSW + block HNSW。	完成
双平台实验	Windows MS-MPI、鲲鹏 direct、鲲鹏 PBS 均有四模式运行日志。	完成

表 7 实验要求与实现证据对照

7 问题与改进方向

更多进程数的扩展性。本报告以 `np=2` 做双平台稳定验证, 已经能证明 MPI 数据流、合并逻辑和 PBS 提交流程正确。但要更系统地分析扩展性, 还应补充 `np=1,2,4,8` 的 sweep, 并在服务器负载较低时重复多次取中位数。预计当进程数增大时, 本地搜索时间会下降, 但候选 gather 和 rank 0 merge 会逐渐变得更明显。

HNSW-on-HNSW 参数。`nprobe_blocks=16` 是偏向高召回的配置, 相当于在 top-level graph 之后仍搜索全部 block。后续可以设置 `nprobe_blocks=1,2,4,8,16`, 画出 Recall@10 与延迟曲线, 找到召回刚刚接近 1.0 的拐点。

更细的负载均衡。当前 base 按连续区间切分, 简单稳定, 但没有根据 inverted list 长度、HNSW 图度数或查询热度做动态均衡。如果某些 rank 的局部图搜索持续更慢, 可以考虑按随机打散、质心均衡或 list 粒度分配来减小尾部延迟。

通信优化。当前所有候选都 gather 到 rank 0。对于更多 rank, 可以改为分层合并: 先在节点内合并, 再跨节点合并; 或者使用 `MPI_Reduce` 的自定义 top-*k* 归约。但在本实验记录中通信/merge 占比很低, 因此这不是当前的首要瓶颈。

8 结论

本实验完成了 ANN 搜索从单机多线程到 MPI 多进程环境的迁移, 并保留 OpenMP 作为 rank 内并行层。IVF-PQ 验证了基础倒排索引迁移; block-HNSW、IVF+HNSW 和 HNSW-on-HNSW 分别覆盖了图索引要求中的 B、A、C 三类方案。双平台结果显示, 相同参数下 Recall 在 Windows 与鲲鹏上完全一致, 说明分布式 top-*k* 合并逻辑正确; 通信与 merge 开销在本验证规模中很小, 主要瓶颈仍是本地 ANN 搜索。

从算法效果看, IVF+HNSW 在 Windows 上延迟最低但召回略低; block-HNSW 在三条路径上都达到 Recall@10=1.00000 且延迟较稳, 是当前最均衡的图索引方案; HNSW-on-HNSW 证明了方案 C 的可行性和高召回能力, 但全 block 探测配置带来了明显延迟代价。整体来看, 本次实现满足了 MPI 划分、混合并行、通信/merge 分析、图索引 A/B/C 和跨平台复现实验的要求。

参考文献

- [1] MPI Forum. *MPI: A Message-Passing Interface Standard*. Version 4.1, 2023.
- [2] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 2011.
- [3] Y. Malkov and D. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE TPAMI*, 2020.
- [4] A. Babenko and V. Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. *CVPR*, 2016.